Dokumentace jazyka Ilang

Obsah

1. Gramatika . . . . . 2.
2. Sémantika . . . . . 5.
   2.1. Typy . . . . . 5.
   2.2. Rozsah platnosti . . . . . 5.
   2.3. Hodnoty a výrazy . . . . . 5.
   2.4. Správa paměti . . . . . 5.
   2.5. Předávání argumentů . . . . . 5.
   2.6. Precedence operátorů . . . . . 6.
   2.7. Omezení . . . . . 6.
3. Psaní programů . . . . . 7.
   3.1. Hello World . . . . . 7.
   3.2. Práce s různými typy . . . . . 7.
   3.3. Deklarace funkcí . . . . . 7.
   3.4. Externí funkce . . . . . 7.
   3.5. Vstup a výstup . . . . . 8.
   3.6. Práce s poli . . . . . 8.
   3.7. Neměnná globální hodnota . . . . . 8.
   3.8. Rekurze . . . . . 9.
   3.9. Cyklus . . . . . 9.
   3.10. Odkazy . . . . . 9.
4. Překladač . . . . . 10.
   4.1. Architektura . . . . . 10.
   4.2. Volací konvence . . . . . 10.
   4.3. Použití . . . . . 10.
   4.4. Rozbor konkrétního programu . . . . . 11.

1. Gramatika

Níže je vypsaná formální gramatika jazyka ve formě ekvivalentí s rozšířenou Backus-Naurovou formou.

program →
  { (*external_declaration* | *function_declaration* | *comment*) }

comment →
  `#` ? any characters ? `\n`

external_declaration →
  `extrn` *type identifier* `(`
    ([*argument* { `,` *argument* } [`,` `…`]] | `…`)
  `)`

function_declaration →
  type identifier `(`
    [*argument* { `,` *argument* }]
  `)` *block_expression*

argument →
  type identifier

block_expression →
  `{` { *expression* `;` } [*expression*] `}`

expression →
  (*return* | *bind* | *assignment* | *value*)

value →
  (*primary* | *binary* | *unary*)

return →
  `return` *value*

bind →
  `let` *identifier* `:` *type* `=` *value*

assignment →
  (*identifier* | *index* | *deref*) `=` *value*

deref →
  `@` *identifier*

binary →
  primary binary_operator value

unary →
  unary_operator primary

index →
  identifier `[` *value* `]`

primary →
(*literal* | *identifier* | *call* | *separated* | *block* | *condition* | *index* | *deref* | *loop* | *make* | *release*)

make →
`make` `(` *basic_type* `,` *value* `)`

release →
`release` `(` *identifier* `)`

loop →
`for` *value block_expression*

literal →
(*basic_literal* | *array_literal*)

basic_literal →
(*int_literal* | *float_literal* | *string_literal* | *bool_literal*)

array_literal →
`[` *value* { `,` *value* } `]`

separated →
`(` *value* `)`

condition →
`if` *value value* [`else` *value*]

identifier →
letter { (*letter* | *digit* | `_`) }

int_literal →
digit { *digit* }

float_literal →
digit { *digit* } `.` *digit* { *digit* }

string_literal →
`“` ? any characters ? `”`

bool_literal →
(`true` | `false`)

type →
(*basic_type* | *array_type* | *slice_type* | *pointer_type*)

basic_type →
(`int` | `bool` | `float` | `string` | `unit`)

pointer_type →
`^` *basic_type*

binary_operator →
(`+` | `-` | `*` | `/` | `==` | `!=` | `<` | `>` | `<=` | `>=` | `<<` | `>>` | `||`)

```
unary_operator →
  ( `-` | `!` | `^` | `@` )

letter →
  ( ? a ... z ? | ? A ... Z ? )

letter →
  ? 0 ... 9 ?
```

2. Sémantika

2.1. Typy

Jazyk je staticky typovaný. Každý výraz má typ určený při překladu.

Základní typy jsou:
- **int** – 64bitové celé číslo se znaménkem
- **float** – 64bitové desetinné číslo, ekvivalent **double** v C
- **bool** – pravdivostní hodnota **true**(1) nebo **false**(0)
- **string** – odkaz na řetězec znaků zakončený nulou
- **unit** – typ bez velikosti, používán pro výrazy bez návratové hodnoty

Složené typy jsou:
- **[N]T** – pole **N** prvků typu **T** alokované na zásobníku kde **N** je přirozené kladné číslo známé při překladu
- **[R]T** – odkaz na pole **R** prvků typu **T** alokované na zásobníku nebo haldě
- **[]T** – odkaz na pole prvků typu **T** alokované na zásobníku nebo haldě
- **^T** – odkaz na hodnotu základního typu **T**

2.2. Rozsah platnosti

Jazyk má pro proměnné jediný rozsah platnosti, a to v těle funkce. Všechny proměnné alokované pomocí **let** jsou přístupné od místa deklarace do konce těla funkce. Externí funkce deklarované pomocí **extrn** jsou dostupné v těle každé funkce nezávisle na pořadí deklarace. Normální funkce jsou dostupné ve vlastním těle(pro podporu rekurze) a v těle všech následujících funkcí.

2.3. Hodnoty a výrazy

Každý výraz vrací hodnotu. Bloky **{ … }** vrací hodnotu posledního výrazu v těle, pokud za ním není středník. V případě, že poslední výraz za sebou středník má, vrací **0**. Podmínka **if** vrací hodnotu větve, která byla vyhodnocena. Cyklus **for** vrací poslední hodnotu těla, nebo **0** pokud tělo neproběhlo ani jednou.

2.4. Správa paměti

Pole(arrays) jsou alokována na zásobníku a jejich velikost musí být známa při překladu. Dynamická pole(slices) jsou buďto odkazy na pole na zásobníku(tak jsou pole předávána do funkcí) nebo jsou alokována na haldě pomocí **make(T, N)**, kde **T** je základní typ a **N** je počet prvků, který nemusí být při překladu známý. Vestavěná funkce **make** je abstrakcí nad funkcí z libc **malloc**. Správa paměti je plně na uživateli, a tak musí alokovanou paměť dealokovat pomocí vestavěné funkce **relase(S)** kde **S** je identifikátor s typem **slice**. Vestavěná funkce **release** je abstrakcí nad funckcí z libc **free**.

2.5. Předávání argumentů

Argumenty jsou předávány hodnotou. Pole a odkazy na pole jsou předávány jako dvojice (**odkaz délka**). Úpravy prvků pole uvnitř funkce se tedy projeví i mimo ni, přiřazení pole ale nikoliv. To jenom upraví hodnotu odkazu a délky v lokální proměnné. Výjimka je pro argumenty s typem pole, kde známe délku při překladu. V tom případě dojde při přiřazení k hodnotě se stejným typem k překopírování prvků.

2.6. Precedence operátorů

Precedence operátorů není přímo definována gramatikou, ale následující tabulkou:

1	||
2	&&
3	==, ≠
4	<, >, ≤, ≥
5	+, -
6	*, /
7	≪, ≫

2.7. Omezení

Logické operátory nepodporují zkrácené vyhodnocení, vždy se vyhodnotí celý výraz. Indexování je povoleno jenom pro hodnoty typu **slice** nebo **array**, ne pro hodnoty typu **string**. Koerce typů je podporována jen pro implicitní převod hodnoty typu **array** na odkaz typu **slice**.

3. Psaní programů

3.1. Hello World

Jednoduchý program vypisující text na standardní výstup:

```
extrn unit printf(string format, ...)

int main() {
  printf("Hello, world!");
  0
}
```

3.2. Práce s různými typy

```
let x: int = 42;
let pi: float = 3.14;
let b: bool = false;
let s: string = "text";
```

3.3. Deklarace funkcí

Jednoduchá funkce vracející součet dvou argumentů:

```
extrn unit printf(string format, ...)

int add2(int a, int b) {
  a+b
}

int main() {
  printf("Hello, world!");
  0
}
```

3.4. Externí funkce

Ukázka použití funkcí z **<math.h>**, při překladu pomocí **gcc** potom nutno poskytnout linker flag **-lm**

```
extrn unit srand(int seed)
extrn int time(int loc)
extrn int rand()

srand(time(0));
let random_number: int = rand();
```

3.5. Vstup a výstup

Pro vstup a výstup se používají funkce ze standardní knihovny C:

```
extrn unit printf(string fmt, ...)
extrn int scanf(string fmt, ...)
extrn int putchar(int c)
extrn int getchar()
```

3.6. Práce s poli

Statické pole fixní velikosti:

```
let arr: [8]int = [1, 2, 3, 4, 5, 6, 7, 8];
arr[3] = 13;
let x: int = arr[3];  # x = 13
```

Dynamické pole alokované na haldě:

```
let n: int = 100;
let arr: [arr_len]int = make(int, n);
arr[0] = 42;
release(arr);
```

Předání pole do funkce a přiřazení délky lokální proměnné:

```
extrn unit printf(string format, ...)

unit print_int_slice_backwards([slice_len]int slice) {
  for slice_len > 0 {
    slice_len = slice_len - 1;
    printf("slice[%d] = %d\n", slice_len, slice[slice_len]);
  }
}

unit main() {
  let n: int = 10;
  let arr: []int = make(int, n);
  print_int_slice_backwards(arr);
  release(arr);
}
```

3.7. Neměnná globální hodnota

Použití funkce jako globální konstanty

```
int max_buffer_size() { 1024 }

unit main() {
  let buffer: []int = make(int, max_buffer_size());
  release(buffer);
}
```

3.8. Rekurze

Využití rekurze pro výpočet faktoriálu:

```
int factorial(int n) {
    if n <= 1 {
        1
    } else {
        n * factorial(n - 1)
    }
}
```

3.9. Cyklus

Využití návratové hodnody cyklu **for** pro iterativní výpočet faktoriálu:

```
let a: int = {
               let n: int = 5;
               let res: int = n;
               for n > 1 {
                 n = n-1;
                 res = res * n
               }
             };
```

3.10. Odkazy

Adresa proměnné se získá operátorem **^**, dereference operátorem **@**:

```
int x: int = 42;
let p: ^int = ^x;
@p = 100;  # x = 100
```

Přečtení čísla z standardního vstupu:

```
extrn int scanf(string format, ...)

int x: int = 0;
scanf("%d", ^x);
```

4. Překladač

4.1. Architektura

Překladač překládá zdrojový kód do GNU assembly pro architekturu x86_64. Skládá se z těchto částí:

- **Lexická analýza** – rozčlenění zdrojového kódu na tokeny
- **Syntaktická analýza** – sestavení abstraktního syntaktickýho stromu (AST) z tokenů
- **Vyhodnocení jmen** – projde strom a propojí identifikátory
- **Vyhodnocení typů** – projde strom a propaguje nahoru typy výrazů
- **Ověření typů** – ověření, jestli typy ve výrazech odpovídají očekávaným
- **Generátor kódu** – projde strom a vygeneruje odpovídající assembly

Výsledný assembly kód je přeložen pomocí GCC do spustitelného souboru.

4.2. Volací konvence

Překladač generuje kód dodržující konvenci System V AMD64 ABI, která se používá na Linuxových systémech. Celočíselné argumenty jsou předávány nejprve šesti registry **%rdi**, **%rsi**, **%rdx**, **%rcx**, **%r8** a **%r9**, argumenty typu **float** nejprve osmi registry **%xmm0**, **%xmm1**, **%xmm2**, **%xmm3**, **%xmm4**, **%xmm5**, **%xmm6** a **%xmm7**. Další argumenty jsou předávány na zásobníku. Před voláním funkcí je zásobník zarovnán na 16 bajtů.

4.3. Použití

Nejlépe skrz **justfile** pomocí programu just.

```
just --list
```

Nebo manuálně:

Přeložení programu **program.ilang**:

```
go run cmd/compiler/main.go -i ./program.ilang -s ./program.s
gcc -no-pie -o program program.s
```

Vypsání tokenů program **program.ilang**

```
go run cmd/compiler/main.go -i ./program.ilang -tk ./program.txt
```

Vypsání AST programu **program.ilang** ve formátu dot a vytvoření obrázku

```
go run cmd/compiler/main.go -i ./program.ilang -tk ./program.dot
dot -Tpng program.dot -o program.png
```

4.4. Rozbor konkrétního programu

Následuje rozbor programu který implementuje buněčný automat rule110.

Celý zdrojový kód je dostupný v **examples/rule110.ilang**

Deklarace dvou externích funkcí:

```
extrn unit printf(string format, ...)
extrn unit scanf(string format, ...)
```

```
assembly:

.extern printf
.extern scanf
```

Deklarace funkce která vypíše stav buněk:

```
unit print_board([slice_len]int board) {
  let idx: int = 0;
  for idx < slice_len {
    printf(if !(board[idx] == 1) { " " } else { "#" });
    idx = idx + 1;
  };
  printf("\n");
}
```

Prolog funkce:

```
assembly:

print_board:
push %rbp
mov %rsp, %rbp
sub $32, %rsp # alokace lokálních proměnných
```

Přesun argumentů funkce do lokálních proměnných:

```
assembly:

mov %rdi, -24(%rbp) # odkaz na pole
mov %rsi, -16(%rbp) # délka pole
mov %rsi, -8(%rbp) # délka pole dostupná v lokální proměnné
```

Přiřazení hodnoty **0** lokální proměnné **idx**:

```
let idx: int = 0;
```

```
assembly:

mov $0, %rax
mov %rax, -32(%rbp)
```

Cyklus **for**:

```
for idx < slice_len {
  ...
};
```

```
assembly:

mov $0, %rax # výchozí návratová hodnota cyklu
.label_1:
push %rax # uložení navratové hodnoty
# binární výraz
mov -32(%rbp), %rax # levá strana - lokální proměnná
push %rax
mov -8(%rbp), %rax # pravá strana - lokální proměnná
mov %rax, %rbx
pop %rax
cmp %rbx, %rax # binární operátor - menší než
setl %al
movzbq %al, %rax
cmp $1, %rax
jne .label_2 # pokud není pravda, skok na konec cyklu
pop %rax # smazání předchozí návratové hodnoty

  ... # tělo cyklu

jmp .label_1
.label_2:
pop %rax # načtení návratové hodnoty
```

Volání externí funkce **printf** s kondicí jako argumentem:

```
printf(if !(board[idx] == 1) { " " } else { "#" });
```

assembly:

```
mov -32(%rbp), %rax # hodnota indexu
push %rax
mov -24(%rbp), %rcx # odkaz na pole
pop %rdx
mov (%rcx, %rdx, 8), %rax # indexování odkazu na pole hodnotou
push %rax
mov $1, %rax # hodnota 1
mov %rax, %rbx
pop %rax
cmp %rbx, %rax # binární operátor - rovnost
sete %al
movzbq %al, %rax
cmp $0, %rax # kondice
sete %al
movzbq %al, %rax
cmp $1, %rax
je .label_3
jmp .label_4
.label_3: # větev if
lea .const_0(%rip), %rax # odkaz na řetězec
jmp .label_5
.label_4:  # větev else
lea .const_1(%rip), %rax # odkaz na řetězec
.label_5:
push %rax
mov 0(%rsp), %rax
mov %rax, %rdi # 1. argument funkce printf
add $8, %rsp # zarovnání zásobníku
mov $0, %rax # počet variadických argumentů
call printf@PLT # volání externí funkce
```

Funkce vracející hodnotu buňky v závislosti na jejím okolí:

```
int rule110(int a, int b, int c) {
    let table: [8]int = [0, 1, 1, 1, 0, 1, 1, 0];
    let idx: int = (a << 2) || (b << 1) || c;
    table[idx]
}
```

Funkce která vypočítá nový stav buňek:

```
unit next_iter([slice_len]int board, [next_len]int next_board) {
  let a: int = 0;
  let b: int = 0;
  let c: int = 0;
  let idx: int = 0;
  for idx < slice_len {
    if idx == 0 {
      a = board[(slice_len - 1)];
    } else {
      a = board[(idx - 1)];
    };
    b = board[idx];
    if idx == (slice_len-1) {
      c = board[0];
    } else {
      c = board[(idx + 1)];
    };
    let val: int = rule110(a,b,c);
    next_board[idx] = val;
    idx = idx+1;
  };
}
```

Funkce která vypočítá a vypíše po **n** sobě jdoucích stavů buněk:

```
unit print_n_iterations([board_len]int board, [next_len]int next_board, int
iters) {
  for iters > 0 {
    let tmp: []int = board;
    next_iter(board, next_board);
    print_board(next_board);
    tmp = board;
    board = next_board;
    next_board = tmp;
    iters = iters - 1;
  }
}
```

Funkce k přečtení čísla od uživatele ze standardního vstupu:

```
int read_number_from_stdin(string prompt) {
  let number: int = 0;
  printf(prompt);
  scanf("%d", ^number);
  number
}
```

Vstupní bod programu:

```
int main() {
  let size: int = read_number_from_stdin("board size: ");
  let board: []int = make(int, size);
  let next_board: []int = make(int, size);
  board[size-1] = 1;
  print_board(board);
  print_n_iterations(board, next_board, size-1);
  release(board);
  release(next_board);
  0
}
```

Příklad výstupu pro **n** = 10:

```
board size: 10
         #
        ##
       ###
      ## #
     #####
    ##   #
   ###  ##
  ## # ###
 ####### #
##     ###
```